

Budget Analysis Agent

Full Technical Architecture, Design Patterns & Interview Preparation Guide

1. Executive Project Summary

The **Budget Analysis Agent** is an enterprise-grade AI financial assistant designed to solve the problem of fragmented, multi-format financial data (PDF bank statements, CSV exports, Excel sheets). It automates transaction ingestion, categorization, relational database persistence, and natural-language financial intelligence (trend analysis, recurring expense detection, and budget optimization).

Core Value Proposition: Uses the **Model Context Protocol (MCP)** to strictly decouple AI reasoning from sensitive filesystem and database operations. The language model operates entirely through standardized, secure MCP tools, ensuring zero direct filesystem/SQL access, local data privacy, and deterministic execution.

2. System Architecture & Component Breakdown

Component	File Location	Core Responsibility & Tech Stack
Agent Orchestrator	agent.py	ClientSession MCP connection, OpenAI function mapping, multi-turn tool-calling loop with NVIDIA NIM (Llama-3.3-70b).
Filesystem MCP Server	mcp_servers/filesystem_server.py	Secure read-only document ingestion (CSV, XLSX, PDF via pdfplumber). Implements strict path-traversal sandboxing.
SQLite MCP Server	mcp_servers/sqlite_server.py	Relational storage owner. Exposes analytical endpoints (query_summary, monthly_comparison, list_recurring_merchants, run_query).
Streamlit Web UI	app.py	Interactive chat web interface with document uploader and event loop synchronization using nest_asyncio.
Database Seeder	seed_db.py	Automated dataset initialization script populating 45 sample multi-month transactions.

3. Key Architectural Patterns for Interviews

A. Model Context Protocol (MCP) Integration

Instead of using monolithic hardcoded tool bindings, the project adopts the open **Model Context Protocol (MCP)**. The agent acts as an **MCP Client** communicating over stdio with two independent, lightweight **MCP Servers** (FastMCP). This enforces clean separation of concerns, language-agnostic tool execution, and modular extensibility.

B. Zero-Direct Access Security Architecture

The AI Model has **zero direct permissions** to open local files or run SQL queries. Every file read is mediated through `filesystem_server.py` which blocks directory traversal attempts (e.g. `../env`). Every database interaction is mediated through `sqlite_server.py` which enforces read-only SELECT validation for ad-hoc queries.

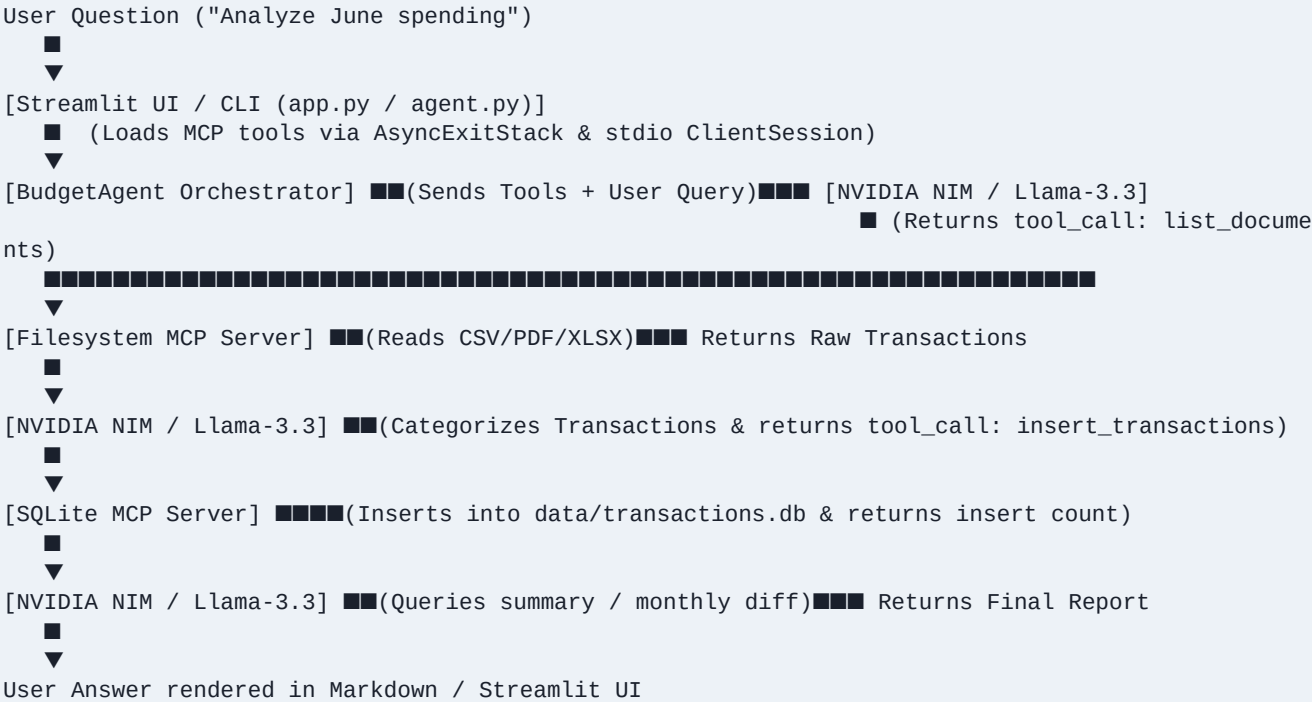
C. Dynamic Function-Calling Re-Act Loop

The Orchestrator dynamically converts FastMCP tool JSON schemas into OpenAI-compatible tool specifications. When the LLM generates a `tool_calls` payload, the orchestrator intercepts it, executes the target tool asynchronously, appends the tool response message, and loops back to the LLM until a final natural-language response is reached.

D. Heuristic Schema Mapping & Subscription Detection

- **Schema Normalization:** Bank exports vary across financial institutions (e.g. Narration vs Particulars vs Description). The filesystem server employs fuzzy column matching.
- **Subscription Algorithm:** SQLite query counts distinct active months per merchant (`COUNT(DISTINCT month) >= min_months`) to detect recurring subscriptions (Netflix, Spotify, Airtel, Rent).

4. End-to-End Execution Flow



5. Top Technical Interview Questions & Answers

Q1: How would you explain this project in 60 seconds to an interviewer?

I built an AI-powered Budget Analysis Agent using Python, Streamlit, SQLite, and the Model Context Protocol (MCP). It ingests multi-format bank statements (PDF, CSV, Excel), categorizes transactions using Llama 3.3 70B via NVIDIA NIM, stores them in SQLite, and delivers natural language financial intelligence. The key innovation is using MCP to decouple LLM reasoning from file/DB access, ensuring data privacy, security sandboxing, and zero direct SQL/filesystem access by the model.

Q2: Why did you choose MCP over standard frameworks like LangChain or LlamaIndex?

Standard frameworks often tightly couple tool definitions with the agent application codebase. MCP provides a standardized, client-server specification. This means our Filesystem and SQLite servers operate as independent microservices that can be reused by any MCP-compliant client (Claude Desktop, custom agents, or IDEs). It also allows us to enforce strict security boundaries at the server process level.

Q3: How do you handle security and SQL injection risks in the run_query tool?

The run_query endpoint is an escape hatch for ad-hoc queries. We implement defense-in-depth: 1. Regex validation ensuring the statement starts strictly with SELECT.

2. Blacklisting dangerous keywords like INSERT, UPDATE, DELETE, DROP, ALTER, PRAGMA.

3. Sandboxed execution against a specific SQLite connection with row factories.

Q4: How does the agent maintain conversation history across tool executions?

In agent.py, we maintain an append-only messages list. When the LLM requests a tool call, we append the assistant's message with tool_calls, execute the tool via the MCP ClientSession, and append a message with role='tool', tool_call_id, and the result string. The full trajectory is passed back in subsequent API calls, preserving multi-turn context.

Q5: What challenges did you face with Streamlit and AsyncIO, and how were they solved?

Streamlit manages its own internal event loop which can conflict with asyncio.run() calls required by MCP's async client sessions. We resolved this by using nest_asyncio.apply() to allow nested event loop execution and created a session-state cached helper get_event_loop() to manage long-lived MCP connections without re-initialization overhead on every Streamlit rerun.

Q6: How would you scale this system for production with millions of transactions?

To scale for production: 1. **Database:** Migrate from SQLite to PostgreSQL with partitioned tables by user_id and month.

2. **Async Queue:** Use Celery / Redis worker queues for heavy document parsing and OCR (Tesseract / AWS Textract for scanned receipts).

3. **Embeddings & RAG:** Use vector search (pgvector) for semantic merchant categorization matching against historical user feedback.

4. **Authentication & Multi-tenancy:** Implement JWT authentication and row-level security (RLS) so users only query their own financial tenant data.

Guide Summary: Prepared for technical software engineering, AI/LLM engineering, and system design interviews.